Project Title:

Database System for a Livestock Farm

SALAWUDEEN SHUKURA ADEBAYO

TABLE OF CONTENT

TITLE PAGE		1
TABLE OF CONTENT		3
ABSTRACT		4
CHAPTER ONE		
1.0	Introduction	5
1.1	Database Management System (DBMS)	6
CHAPTER TWO		
2.0	Literature Review	7
2.1	Database Management Systems	7
2.2	Relational Database Model	7
2.3	Database Design Methodology	8
2.4	Database Systems in Livestock Management	9
CHAPTER THREE		
3.0	Methodology	10
3.1	Overview of Database Design Methodologies	10
3.2	Chosen Methodology	10
3.3	Conceptual Design; Entity Relationship Diagram	11
3.4	Database Summary	12
3.5	Logical Design; Tabular Structure	12
CHAPTER FOUR		
4.0	Implementation and Result discussion	16
4.1	Implementation	16
4.1.1	Choice of Database Management System (DBMS)	16
4.1.2	Database Creation	16
4.1.3	Populating the Database with sample data	18
4.1.4	Testing the Database	19
4.1.5	Tools used	20
4.2	Result Discussion	20
4.2.1	Animals without health record	20
4.2.3	Housing Utilization	22
4.2.4	Top customers by transaction value	22
4.2.5	Breeding Pairs in Progress	23
4.2.6	Summary	24
CHAPTER FIVE		
5.0	Conclusion	25

ABSTRACT

This project develops a relational database system to improve livestock factory management. Many farms rely on manual records, which cause errors and inefficiency. Using MySQL and Entity Relationship (ER) modeling, the system organizes nine interrelated tables covering farms, housing, animals, staff, health records, feeding, customers, transactions, and breeding. The database apply integrity with primary keys, foreign keys, and constraints, ensuring reliable data storage. Testing confirmed that rules such as preventing duplicate animal tags and preserving breeding lineage work correctly. Sample queries demonstrated practical uses like tracking feed costs, monitoring housing utilization, identifying animals without health records, and managing customer transactions. Centered in real Nigerian livestock data, the system provides a credible, efficient, and scalable solution for modern livestock factory operations.

CHAPTER ONE

1.0 Introduction

Livestock farming has continued to play a significant role in the agricultural sector of Nigeria, contributing immensely to food security, income generation, and rural livelihood. Despite this importance, many livestock farms, including small and medium-scale operations, still rely on manual methods such as paper records and ordinary spreadsheets to manage vital information about their animals. This approach, though familiar, is often characterized by errors, loss of records, duplication of data, and difficulty in retrieving information when it is most needed. For instance, a farm manager may find it difficult to quickly determine which animals are due for vaccination, or how much has been spent on feeding in a given month, simply because the relevant records are scattered or poorly organized.

It is against this background that this project became necessary. The aim of this project is to design and implement a relational database system for the management of livestock, using sound database design principles taught in this course. The system, referred to as the Livestock Management Database models a realistic livestock operation similar to what is obtainable at Major farms in Nigeria

The database was designed to capture and manage the following core aspects of livestock operations:

- Farms and the various Housing unit located within them
- Individual animals, including cattle, goats, sheep, and poultry, and their bio-data
- Staff members responsible for the care and management of the animals
- Health records, covering vaccinations, treatments, and routine checkups
- Feeding activities and the cost implication of feeding the animals
- Customers who purchase from or sell animals to the farm

- Sales and purchase transactions
- Breeding events and the lineage of animals, including the sire, dam, and offspring involved

1.1 DATABASE MANAGEMENT SYSTEM (DBMS)

MySQL was adopted as the Database Management System (DBMS) for the implementation of this project. This choice was informed by its wide usage, availability of documentation, ease of use, and its adequacy for handling relational databases of this nature and scale. In all, the database comprises nine (9) interrelated tables, structured around the ANIMAL entity as the central table, with relationships properly enforced using primary keys, foreign keys, cascading rules, and constraints to ensure that the integrity of the data is maintained at all times.

This report is organized to reflect the different stages involved in the design and implementation of the database. Following this introduction, Section 2 discusses the Methodology adopted for the design of the database; Section 3 presents the Implementation, detailing how the database was built and tested; Section 4 discusses the Results obtained from testing the system, including sample queries; while Section 5 concludes the report with a summary of the work carried out.

CHAPTER TWO

2.0 Literature Review

2.1 Database Management Systems

A Database Management System (DBMS) as discussed in class is a software developed to facilitate the storage, organization, retrieval, and management of data in a structured manner. Rather than storing information in separate files or spreadsheets, a DBMS provides a centralized environment in which data can be created, updated, queried, and secured while maintaining consistency and integrity. As emphasized during classroom discussions, one of the primary motivations behind the development of DBMS was to overcome the limitations of traditional file-based systems, such data redundancy, inconsistency, and poor data sharing.

Several Database Management Systems are available today, including Oracle Database, Microsoft SQL Server, PostgreSQL, SQLite, and MySQL.

2.2 Relational Database Model

According to understanding from class, relational model organizes data using tables (relations) consisting of rows and columns.

The class notes similarly emphasized that the strength of the relational model lies not merely in storing data, but in maintaining consistency through integrity constraints. **Primary keys** uniquely identify each record, **foreign keys** establish relationships between tables, while constraints such as UNIQUE, CHECK, and referential actions ensure that only valid data is stored within the database.

The livestock management problem considered in this project fits to the relational model. Real-world entities such as farms, animals, staff members, customers, and health records can each be

represented as individual tables, while their interactions can be modeled through carefully defined relationships. This makes the relational approach particularly suitable for maintaining accurate operational records while supporting efficient querying and reporting.

2.3 Database Design Methodologies

Developing an effective database requires more than simply creating tables; it requires a systematic design methodology capable of accurately modeling the real-world domain. Various methodologies have been proposed in the literature, each approaching database development from a different perspective.

The methodology adopted in the classroom, Entity–Relationship **(ER) Modeling**, which begins:

- identifying entities
- their attributes
- and the relationships that exist among them

before translating the resulting conceptual model into relational tables. Another approach is **Normalization-Driven Design**, in which designers begin with a comprehensive collection of attributes before progressively decomposing them into well-structured tables through the application of the normal forms. More recently, **Agile or Iterative Schema Design** has gained popularity in software development environments, allowing the database structure to evolve alongside changing application requirements.

As discussed during lectures, no single methodology is universally superior; rather, the choice depends on the complexity of the problem, the availability of requirements, and the objectives of the project.

2.4 Database Systems in Livestock Management

The management of livestock operations involves the continuous recording of information relating to animals, housing, feeding, breeding, health management, staff activities, and commercial transactions. Traditionally, many farms have maintained these records using paper files or simple spreadsheet applications. Although such methods may initially appear adequate, they become increasingly difficult to manage as the size of the livestock operation grows.

Several studies on agricultural information systems have shown that computerized databases improve the efficiency of farm management by enabling rapid retrieval of information, reducing duplicate records, and supporting better decision-making. A well-designed livestock database allows farm managers to monitor vaccination schedules, track breeding history, record feed consumption, manage customer transactions, and generate reports that would otherwise require considerable manual effort.

The usefulness of such systems extends beyond record keeping. Because all operational information is stored within a centralized database, managers are able to analyse trends, monitor operational costs, identify animals requiring medical attention, and evaluate farm performance over time. These capabilities illustrate why database systems have become an increasingly important component of modern livestock management.

CHAPTER THREE

3.0 Methodology

3.1 Overview of Database Design Methodologies

Three commonly used approaches were reviewed and discussed among group members:

Entity–Relationship (ER) Modeling: a top-down approach in which real-world objects (entities), their properties (attributes), and the associations between them (relationships) are identified and diagrammed before any tables are created.

Normalization-Driven (Bottom-Up) Design: an approach that begins from a complete list of attributes, which are then progressively grouped and split into tables by applying normal forms (1NF, 2NF, 3NF) to eliminate redundancy.

Agile/Iterative Schema Design: an approach where the schema evolves incrementally alongside development, with tables added or modified as new requirements emerge.

Each of these was weighed against the nature of the project, the skillset/knowledge of the group and the time available to the group.

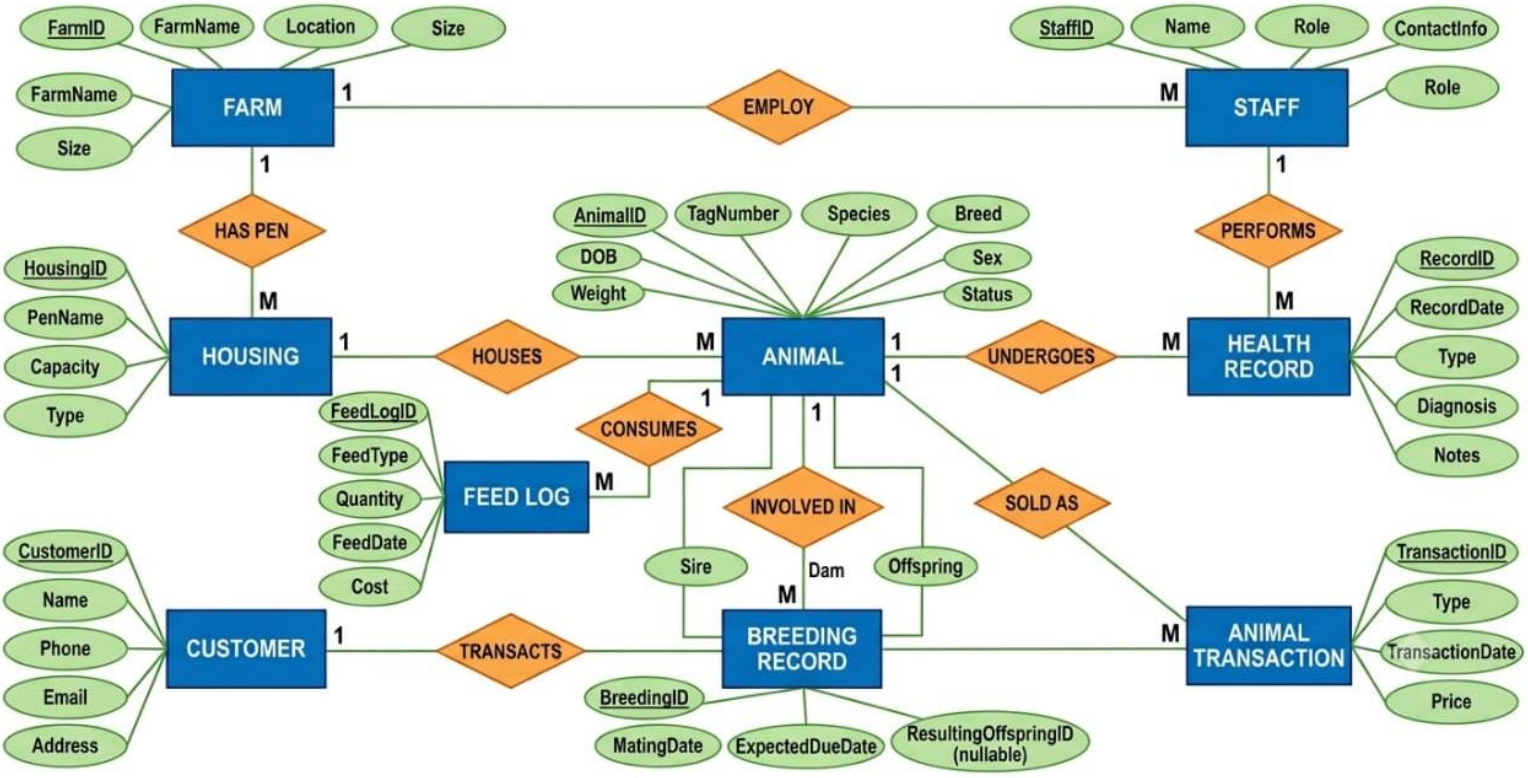
3.2 Chosen Methodology

After deliberation, the group settled on **Entity–Relationship (ER) Modeling**. This choice was informed by the familiarity of the group with this method based on class room activities and the fact that the livestock management domain consists of clearly identifiable real-world objects; farms, pens, animals, staff, health records, and so on with fairly well-defined relationships between them, making ER modeling a natural fit. Building the ER diagram first also allowed the group to visualize and agree on these relationships as a team before any SQL was written, which

proved useful later when the group needed to revisit the design. Once the conceptual model was agreed upon, it was translated into a logical (tabular) schema.

3.3 Conceptual Design; Entity–Relationship Diagram

The conceptual model was developed using Chen notation, where rectangles represent entities, ovals represent attributes (with primary keys underlined), and diamonds represent relationships, with cardinality (1 or M) indicated on each connecting line. The final Livestock Management Database consists of nine (9) entities connected through nine (9) relationships, including one self-referencing relationship (BREEDING RECORD to ANIMAL) that models the three roles an animal can play in a breeding event: Sire, Dam, and Offspring.



3.4 Database Summary

The Livestock Management Database, as finalized, consists of the following:

- **Number of Tables**: 9 (FARM, HOUSING, STAFF, ANIMAL, HEALTH_RECORD, FEED_LOG, CUSTOMER, ANIMAL_TRANSACTION, BREEDING_RECORD)
- **One-to-Many (1:M) Relationships**: 8; FARM–HOUSING, FARM–STAFF, PEN–ANIMAL, STAFF–HEALTH_RECORD, ANIMAL–HEALTH_RECORD, ANIMAL–FEED_LOG, ANIMAL–ANIMAL_TRANSACTION, CUSTOMER–ANIMAL_TRANSACTION
- **Self-Referencing Relationship**: 1; BREEDING_RECORD links back to ANIMAL through three foreign keys (SireID, DamID, OffspringID)

3.5 Logical Design; Tabular Structure

The ER diagram in Figure 1 was translated into the relational schema presented in full, with each table listing its columns, data types, and constraints, including primary keys (PK) and foreign keys (FK).

1. FARM

Column	Data Type / Constraint	Description
FarmID	INT (PK, AUTO_INCREMENT)	Unique identifier for each farm
FarmName	VARCHAR(100)	Name of the farm
Location	VARCHAR(150)	Physical location/address
Size	DECIMAL(10,2)	Land area in hectares

2. HOUSING

Column	Data Type / Constraint	Description
HousingID	INT (PK, AUTO_INCREMENT)	Unique identifier for each pen
FarmID	INT (FK → FARM)	Farm the pen belongs to
PenName	VARCHAR(50)	Identifier/name of the pen
Capacity	INT	Maximum number of animals the pen can hold
Type	VARCHAR(50)	Barn, paddock, coop, etc.

3. STAFF

Column	Data Type / Constraint	Description
StaffID	INT (PK, AUTO_INCREMENT)	Unique identifier for each staff member
FarmID	INT (FK → FARM)	Farm the staff member works at
Name	VARCHAR(100)	Full name
Role	VARCHAR(50)	Vet, handler, manager, etc.
ContactInfo	VARCHAR(100)	Phone or email

4. ANIMAL

Column	Data Type / Constraint	Description
AnimalID	INT (PK, AUTO_INCREMENT)	Unique identifier for each animal
HousingID	INT (FK → PEN)	Pen currently housing the animal
TagNumber	VARCHAR(20), UNIQUE	Physical ear/leg tag identifier

Species	VARCHAR(50)	e.g. Cattle, Goat, Poultry
Breed	VARCHAR(50)	Breed of the animal
DOB	DATE	Date of birth
Sex	ENUM('M','F')	Sex of the animal
Weight	DECIMAL(6,2)	Most recent recorded weight (kg)
Status	ENUM('Active','Sold','Deceased')	Current status of the animal

5. HEALTH_RECORD

Column	**Data Type / Constraint**	**Description**
RecordID	INT (PK, AUTO_INCREMENT)	Unique identifier for each health record
AnimalID	INT (FK → ANIMAL)	Animal the record belongs to
StaffID	INT (FK → STAFF)	Staff member/vet who attended to the animal
RecordDate	DATE	Date of the health event
Type	VARCHAR(50)	Vaccination, treatment, or checkup
Diagnosis	VARCHAR(150)	Diagnosis or condition observed
Notes	TEXT	Additional remarks

6. FEED_LOG

Column	**Data Type / Constraint**	**Description**
FeedLogID	INT (PK, AUTO_INCREMENT)	Unique identifier for each feed entry
AnimalID	INT (FK → ANIMAL)	Animal that was fed
FeedType	VARCHAR(50)	Type of feed given
Quantity	DECIMAL(6,2)	Quantity fed, in kg
FeedDate	DATE	Date the feeding occurred

Cost	DECIMAL(8,2)	Cost of the feed

7. CUSTOMER

Column	**Data Type / Constraint**	**Description**
CustomerID	INT (PK, AUTO_INCREMENT)	Unique identifier for each customer
Name	VARCHAR(100)	Full name of the buyer or seller
Phone	VARCHAR(20)	Contact phone number
Email	VARCHAR(100)	Contact email address
Address	VARCHAR(150)	Physical address

8. ANIMAL_TRANSACTION

Column	**Data Type / Constraint**	**Description**
TransactionID	INT (PK, AUTO_INCREMENT)	Unique identifier for each transaction
AnimalID	INT (FK → ANIMAL)	Animal involved in the transaction
CustomerID	INT (FK → CUSTOMER)	Buyer or seller involved in the transaction
Type	ENUM('Sale','Purchase')	Whether the animal was sold or purchased
TransactionDate	DATE	Date of the transaction
Price	DECIMAL(10,2)	Transaction amount

9. BREEDING_RECORD

Column	**Data Type / Constraint**	**Description**
BreedingID	INT (PK, AUTO_INCREMENT)	Unique identifier for each breeding event
SireID	INT (FK → ANIMAL)	Father animal

DamID	INT (FK → ANIMAL)	Mother animal
MatingDate	DATE	Date of mating
ExpectedDueDate	DATE	Expected date of birth
OffspringID	INT (FK → ANIMAL, nullable)	Resulting offspring, once born

CHAPTER FOUR

4.0 Implementation and Result discussion

4.1 Implementation

4.1.1 Choice of Database Management System (DBMS)

The choice of MySQL as the Database Management System (DBMS) for this project was arrived at as a collective decision by the group. Several DBMS options were considered, but MySQL was eventually settled on based on the general familiarity of group members with the system, as well as its accessibility, since it is free and open-source. Being a widely taught and commonly used DBMS, most members of the group already had some level of exposure to MySQL, which made it a practical and convenient choice for the group to work with within the time available for the project.

MySQL Workbench was used as the primary tool for designing, implementing, and managing the database throughout the project. Its graphical interface made it easier for the group to create tables, define relationships, execute SQL scripts, and visually inspect the database structure during development and testing.

4.1.2 Database Creation

Initially, the group had arrived at eight (8) tables covering the core entities identified during the conceptual design stage: FARM, HOUSING, STAFF, ANIMAL, HEALTH_RECORD, FEED_LOG, ANIMAL_TRANSACTION, and BREEDING_RECORD. However, on closer

examination of the ANIMAL_TRANSACTION table, it was observed that the buyer or seller involved in each transaction was being stored as a plain text field (PartyName). The group identified this as a design weakness: the same customer transacting with the farm on multiple occasions would have their name typed in repeatedly, with no guarantee of consistency, and no way to store additional information about them, such as phone number or address, or to reliably track how much business had been done with a particular customer over time.

After deliberating on this, the group agreed to introduce a ninth table, CUSTOMER, to properly separate customer information from the transaction record itself. This decision reflected a deliberate application of normalization principles discussed in the Methodology section, rather than an arbitrary addition, and it is this refinement that brought the final table count to nine (9).

With the entities finalized, the tables were created using the CREATE DATABASE and CREATE TABLE statements in MySQL Workbench, following the logical design earlier presented in Section 2. Because MySQL requires that a referenced table already exists before a foreign key can be created against it, the tables were created in a specific order: tables with no dependencies on any other table (such as FARM and CUSTOMER) were created first, followed by tables that reference them (such as HOUSING, STAFF, and ANIMAL_TRANSACTION), and finally tables with multiple dependencies (such as BREEDING_RECORD, which references ANIMAL three separate times).

Each table definition specified the appropriate data types, primary keys, and, where necessary, foreign key constraints with defined referential actions (CASCADE, SET NULL, or RESTRICT) to control what happens when related records are updated or deleted. For instance, deleting a FARM record automatically deletes its associated PEN and STAFF records (ON DELETE CASCADE), while an attempt to delete an ANIMAL that has breeding history is deliberately blocked (ON DELETE RESTRICT) to preserve lineage records. A CHECK constraint was also

implemented on the BREEDING_RECORD table to ensure that an animal cannot be recorded as both the sire and dam of the same breeding event.

4.1.3 Populating the Database with Sample Data

Having created the tables, the next task was to populate the database with sample data that would allow the system to be meaningfully tested and demonstrated. At the onset, the group considered simply generating arbitrary values, random names, made-up breeds, and round-number prices as is commonly done in student database projects. However, it was agreed that this approach would weaken the credibility of the project, since any reasonable questioning during presentation (for instance, "why does this breed cost this much?") would be difficult to defend convincingly.

The group therefore made a conscious decision to ground the sample data in real-world information relevant to livestock farming in Nigeria, rather than relying on arbitrary placeholder values. This involved a short research process, after which the following was incorporated into the data:

- Genuine cattle, goat, and sheep breeds commonly reared and traded in Nigeria, such as White Fulani, Sokoto Gudali, Red Sokoto, and Yankasa
- Actual vaccines administered by Nigerian veterinary services, including FMD, Anthrax, and CBPP vaccines for cattle, PPR vaccine for goats and sheep, and Newcastle Disease vaccine for poultry
- Realistic transaction prices reflective of current (2025/2026) Nigerian livestock market price ranges, rather than round or arbitrary figures

This decision meant that when the database was later queried during testing (see Section 4), the results obtained — such as total feed cost per pen or revenue generated per customer — were not just syntactically correct, but also meaningful and defensible figures that could pass scrutiny

during the group's presentation. The full sample data script used to populate the database is provided in Appendix B.

4.1.4 Testing the Database

Creating the tables and writing the constraints was only one part of the implementation; the group was conscious that a constraint written in SQL is only useful if it can be shown to actually work. For this reason, before considering the database complete, the group carried out a deliberate round of testing aimed specifically at breaking the rules that had been built into the schema, in order to confirm that the database would reject invalid operations and correctly handle dependent records.

Rather than testing every constraint in the schema, the group selected five tests considered most representative of the different categories of integrity built into the database: a domain-specific business rule (the CHECK constraint), referential integrity (foreign keys), entity integrity (a UNIQUE constraint), and the two referential actions used most extensively in the schema (CASCADE and RESTRICT).

Summary of Database Constraint Tests

Test	Rule Being Tested	Expected Outcome	Actual Outcome
Insert a breeding record with the same animal as both Sire and Dam	CHECK constraint	Rejected	Rejected
Insert a transaction referencing a non-existent AnimalID	Foreign key constraint	Rejected	Rejected
Insert an animal with a duplicate TagNumber	UNIQUE constraint	Rejected	Rejected
Delete a Farm that has associated	ON DELETE	Pens deleted	Cascaded

Pens	CASCADE	automatically	
Delete an Animal with breeding history	ON DELETE RESTRICT	Deletion blocked	Blocked

All five tests produced the expected results. This gave the group confidence that the database does not merely store data, but actively enforces the rules guiding a real livestock operation — for instance, that an animal cannot biologically be its own parent, that historical breeding records are protected from accidental deletion, and that removing a farm sensibly clears out the pens that no longer have anywhere to exist.

4.1.5 Tools Used

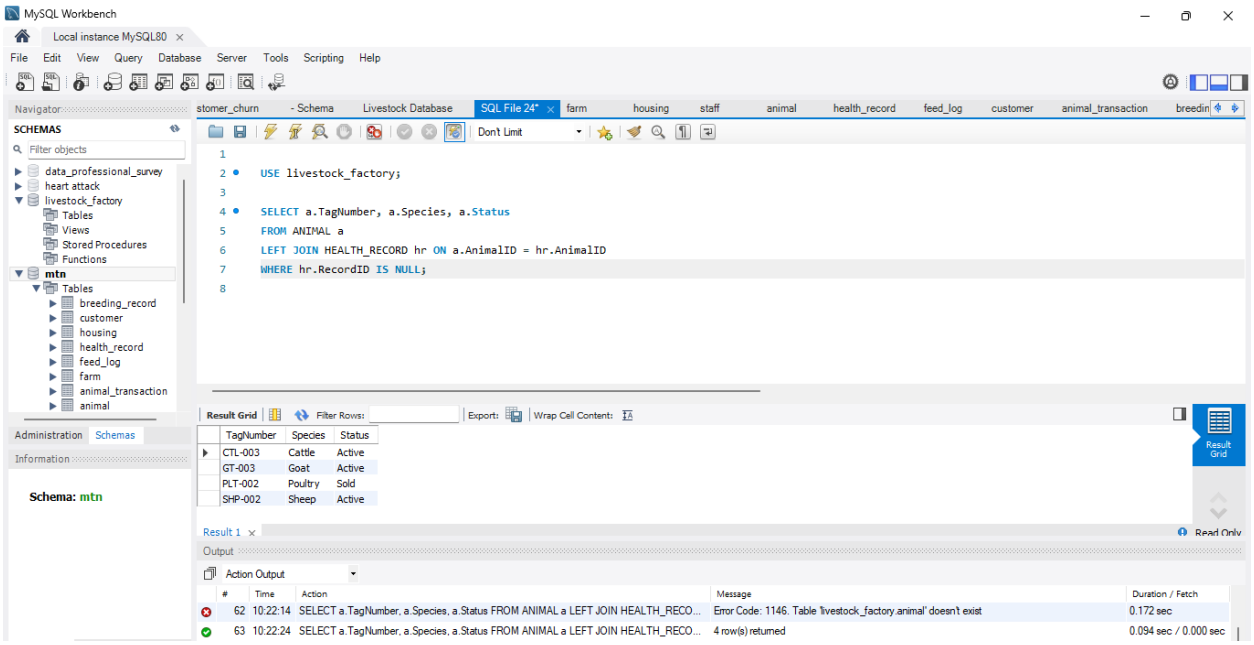
Tool	Purpose
MySQL	Database Management System (DBMS) used to implement the database
MySQL workbench	Designing the schema, executing SQL scripts, and managing the database
SQL (DDL and DML)	Creating tables, defining constraints, and inserting/querying data

4.2 Result Discussion

After the database had been fully implemented, populated, and tested for constraint enforcement (as discussed in Section 3), the group proceeded to run a series of queries against the database to demonstrate that it could actually answer meaningful, real-world questions about the livestock operation. This section discusses a selection of these queries and what their results reveal.

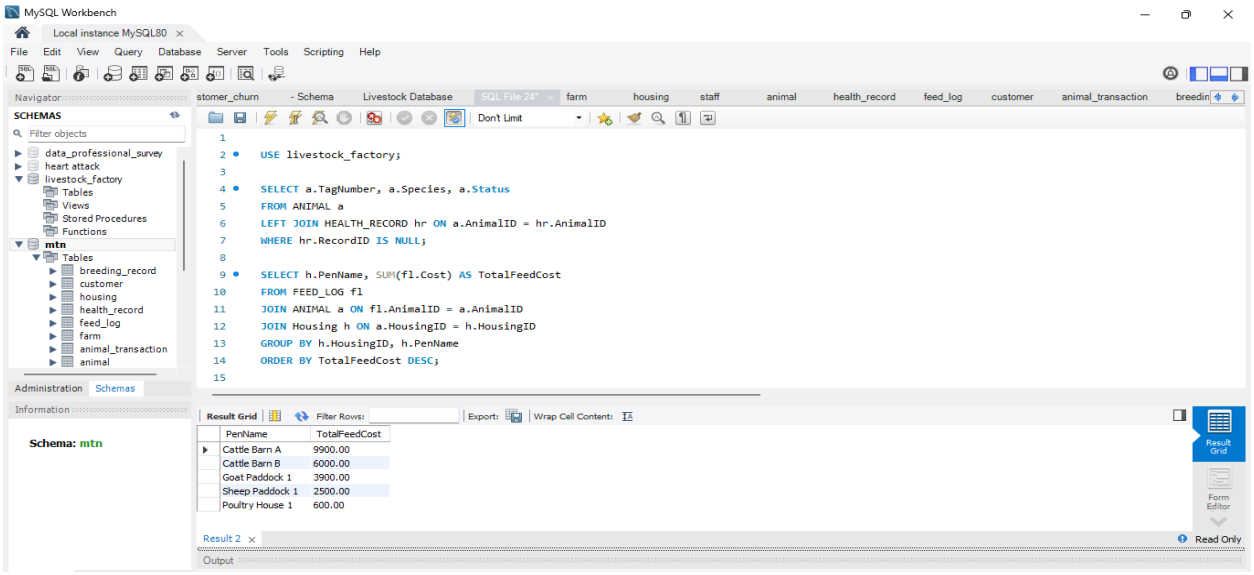
4.2.1 Animals Without Health Records (Compliance Check)

One of the more revealing queries run against the database was one designed to identify animals that had never received any form of health attention:



4.2.2 Total Feed Cost per Housing

To assess where feeding costs were concentrated across the farm, the following query was run:

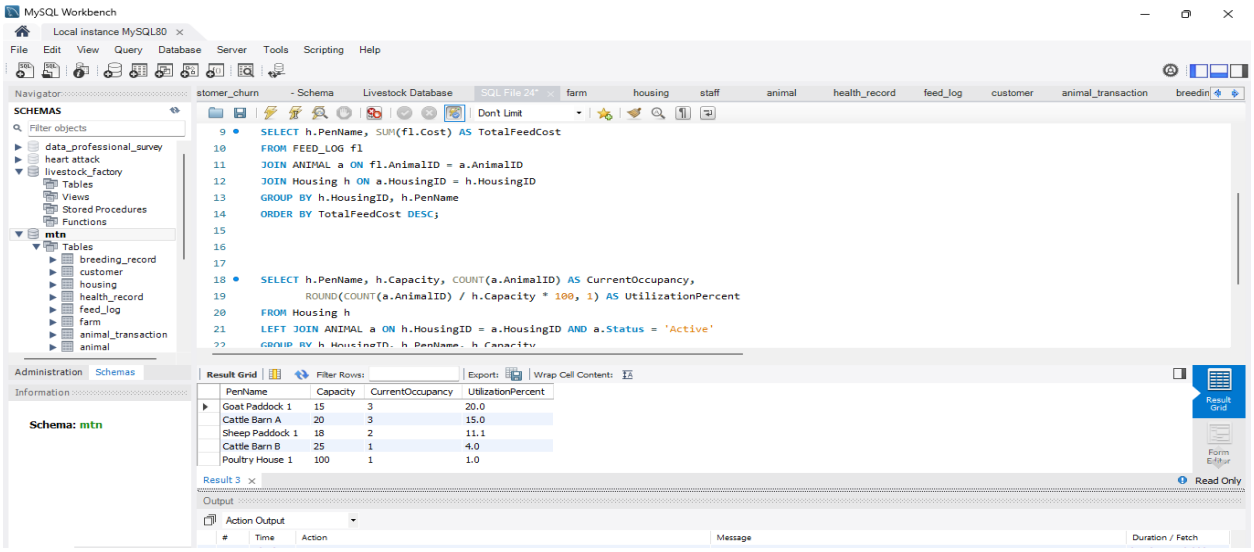


As expected, Cattle Barn A recorded the highest feed cost, which makes sense given that cattle consume considerably more feed than goats, sheep, or poultry, and this pen houses the highest number of actively fed cattle in the dataset. A result such as this would ordinarily assist a farm

manager in budgeting and identifying which sections of the farm are the most expensive to maintain.

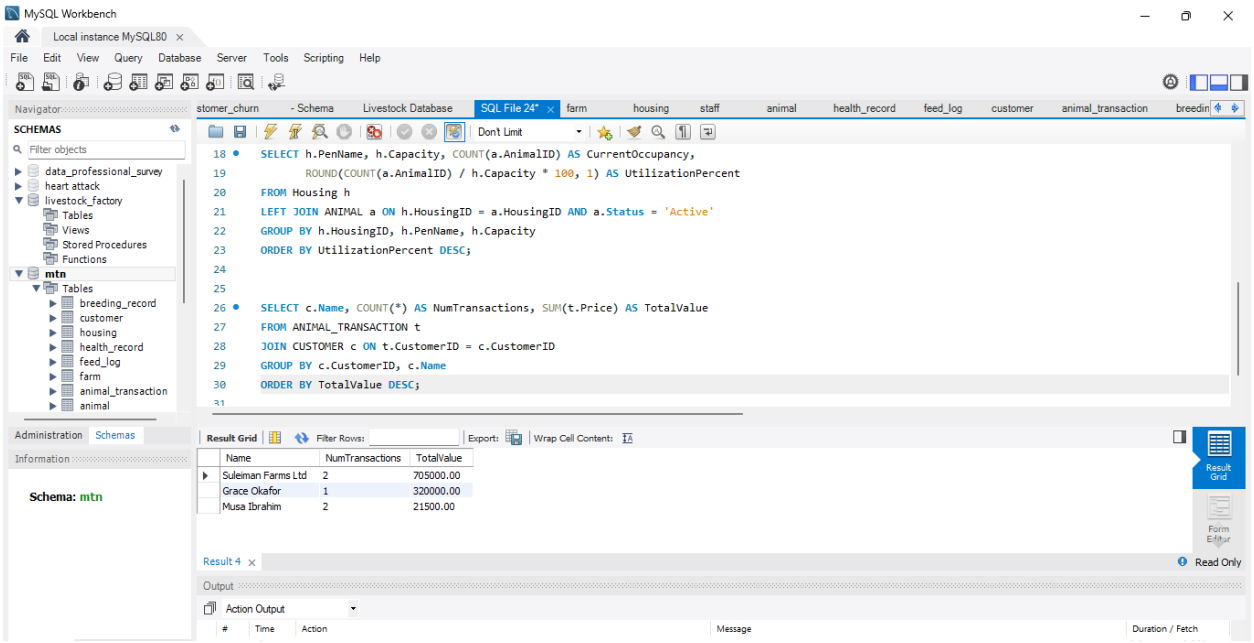
4.2.3 Housing Utilization

The database was also queried to check how well pens were being utilized relative to their stated capacity:



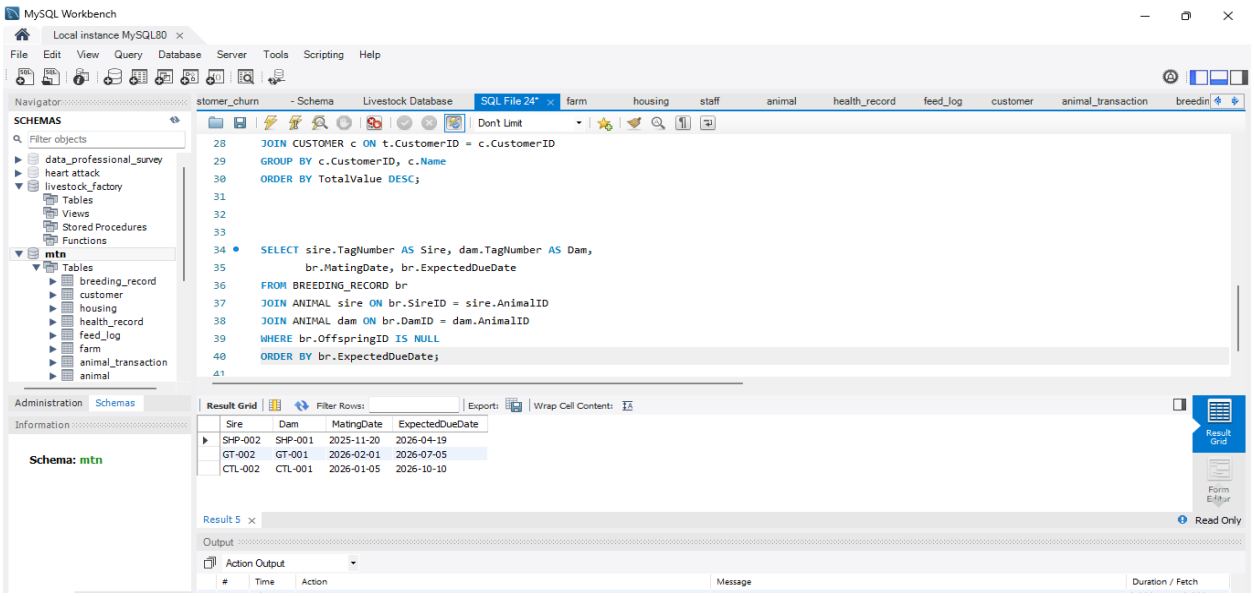
4.2.4 Top Customers by Transaction Value

Since the CUSTOMER table was introduced specifically to enable proper tracking of repeat buyers and sellers, the group tested this directly:



4.2.5 Breeding Pairs in Progress

Finally, to demonstrate the self-referencing relationship discussed extensively in the Methodology section, the group queried ongoing breeding pairs:



4.2.6 Summary

Taken together, these results demonstrate that the Livestock Management Database is not just a structurally sound schema, but a functionally working system capable of answering realistic operational questions — from compliance monitoring and cost analysis to customer relationship tracking and breeding management — using nothing more than standard SQL joins, aggregations, and filters.

CHAPTER FIVE

5.0 Conclusion

This project set out to design and implement a relational database system for the management of livestock operations, using sound database design principles as taught in the course of this study. Over the course of the project, the group progressed through every stage of the database development lifecycle from studying available database design methodologies, through conceptual (ER) design, logical/tabular design, actual implementation in MySQL, population with realistic sample data, and finally rigorous testing of the system's constraints and query capabilities.

Several important outcomes were achieved in the course of this work. The group successfully modeled a livestock operation consisting of interrelated entities; farms, housing units, staff, animals, health records, feeding activity, customers, transactions, and breeding records capturing not just simple one-to-many relationships, but also a self-referencing relationship (Sire, Dam, and Offspring all referencing the same ANIMAL table) that required deeper application of relational design principles than a typical student project of this nature would demand. The group also demonstrated, through deliberate testing, that the constraints defined in the schema (CHECK constraints, foreign keys, and cascading rules) genuinely enforce the business rules of a livestock operation, rather than existing merely as declarations with no real effect.

Beyond the technical implementation, the group made a conscious decision to ground the project in real-world Nigerian livestock data genuine breed names, actual veterinary vaccines, and realistic market prices rather than relying on arbitrary placeholder values, which added a layer of credibility and defensibility to the system that a purely synthetic dataset would not have provided. Similarly, the group's willingness to revisit and refine the design mid-project, such as introducing the CUSTOMER table after identifying a normalization weakness in the original

transaction structure, reflects the kind of iterative thinking that database design, in practice, actually requires.

The project was not without its limitations. The sample data used, while grounded in real-world reference values, remains illustrative rather than drawn from an actual operating farm, given that genuine farm-level operational data of this kind is not publicly available. The database, as implemented, also does not include a user interface or application layer, being limited in scope to the database design and query layer as required by the assignment. Future extensions of this work could include the development of a front-end application (web or mobile) to allow non-technical farm staff to interact with the database directly, as well as the incorporation of additional features such as automated vaccination reminders or financial reporting dashboards built on top of the queries already demonstrated in Section 4.

In all, this project provided the group with practical, hands-on experience in the full database design process from theory to a working, tested implementation and reinforced the importance of sound methodology, data integrity, and iterative refinement in building systems that are not just technically correct, but genuinely useful.